

01 Problem & Approach

Retailers need to know which of their products a competitor also sells, and at what price, to compete on pricing. This system resolves that as an **entity-resolution problem**: given two independent product catalogs (Abt.csv, Buy.csv — the standard Abt-Buy benchmark, 1,081 and 1,092 products, no shared IDs) with no clean join key, determine which pairs refer to the same physical product, then layer competitive pricing analysis on top.

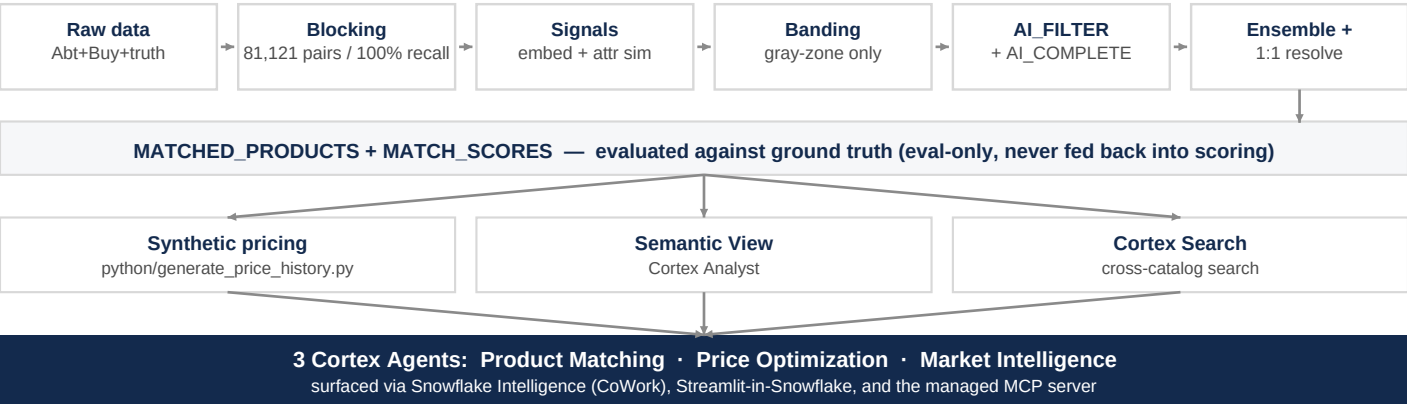
Differentiator: multi-strategy ensemble matching

Most naive approaches run one embedding-similarity pass and threshold it. This system instead combines four independent signals into one explainable score **and rationale** per candidate pair:

01 Blocking	02 Semantic similarity	03 Structured attributes	04 LLM adjudication
Cheap candidate generation — avoids running AI on all ~1.18M possible pairs.	AI_EMBED (once per product) + VECTOR_COSINE_SIMILARITY.	AI_EXTRACT pulls brand/model number; compared via EDITDISTANCE.	Cost-gated: only ambiguous pairs get AI_FILTER + AI_COMPLETE verdict + rationale.

A weighted combination produces final_confidence (MATCH >= 0.70 / REVIEW 0.40–0.70 / NO_MATCH otherwise), followed by a greedy 1:1 conflict-resolution pass (Python stored proc) — no product is claimed twice.

02 Pipeline



03 Three Agents, Three Distinct Techniques

Product Matching Agent	Purpose: Explain/inspect specific matches; ad hoc cross-catalog search. Tool: cortex_search + EXPLAIN_MATCH proc (live re-explanation) Functions: AI_EXTRACT, AI_FILTER, AI_COMPLETE
Price Optimization Agent	Purpose: Per-pair pricing recommendation. Tool: RECOMMEND_PRICE proc — deterministic rule engine; LLM only phrases the justification Functions: AI_COMPLETE (justification only)
Market Intelligence Agent	Purpose: Category/trend narratives. Tool: MARKET_TREND_SUMMARY proc wrapping AI_AGG over AI_CLASSIFY-derived labels Functions: AI_AGG, AI_CLASSIFY

All three sit on one semantic view (ABT_BUY_SEMANTIC_VIEW) so the same objects back the Cortex Agents, the Snowflake Intelligence/CoWork chat surface, the Streamlit dashboards, and the MCP server — no duplicated build effort.

04 Synthetic Data Disclosure

IMPORTANT — pricing history in this demo is simulated

The Abt-Buy dataset has no time-series pricing: only a single point-in-time price exists per product, missing on 46–62% of rows. generate_price_history.py fabricates an 8–12-week weekly series per matched pair per retailer (real price where available, else bootstrapped, plus random-walk drift and occasional promo/undercut events). Every row is tagged is_synthetic = TRUE end-to-end and shown as a persistent banner in the dashboard. Product identities and matches are computed entirely from the real dataset — only the price history is simulated.

05 Results

PRECISION	RECALL	F1 SCORE	BLOCKING RECALL
97.9%	86.6%	0.919	100%

Predicted MATCH: 970 · True positives: 950 · False positives: 20 · False negatives: 147 · Review queue: 83

Measured live via V_MATCHING_ACCURACY against the real abt_buy_perfectMapping.csv ground truth. Precision is intentionally the priority — a MATCH label is right 98% of the time. Ambiguous pairs land in a separate REVIEW tier rather than being forced into MATCH or NO_MATCH, feeding a human-in-the-loop review queue instead of a false claim of certainty.

06 Tech Stack

Snowflake	Cortex AI SQL	Cortex Search	Cortex Analyst	Cortex Agents	Snowflake Intelligence / CoWork
Managed MCP Server	Streamlit-in-Snowflake	Python	SQL		

Every match is explainable. Every price is justified. Every trend is narrated.

Built entirely on Snowflake Cortex — AI SQL, Search, Analyst, Agents, and a managed MCP server.